МОЛДАВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет математики и информатики

Департамент Информатики

Лабороторная работа номер 3:

Основы работы с массивами, функциями и объектами в JavaScript

Проверил: профессор, др. N. Nartea

Выполнил: Боаги Никита IA2503

Кишинев 2026г

Цель работы

Изучить основы работы с массивами и функциями в JavaScript, применяя их для обработки и анализа транзакций.

```
function getUniqueTransactionTypes(transactions) {
    return  [...new Set(transactions.map (t => t.transaction_type))];
}
```

- Берёт все транзакции и создаёт массив только с типами транзакций (debit, credit и т.д.).
- Set удаляет все повторяющиеся значения.
- [...] превращает Set обратно в массив.
- Возвращает массив уникальных типов транзакций.

```
function calculateTotalAmount(transactions){
    return transactions.reduce((sum, t) => sum + t.transaction_amount, 0);
}
```

- Суммирует все суммы транзакций (transaction_amount).
- Используется reduce, чтобы пройтись по каждому элементу и накопить сумму.
- Возвращает общую сумму всех транзакций.

```
function getTransactionByType(transactions, type){
    return transactions.filter (t => t.transaction_type ===type);
}
```

- Фильтрует транзакции по конкретному типу (debit или credit).
- Возвращает массив транзакций указанного типа.

```
function getTransactionsInDateRange(transactions, startDate, endDate) {
  return transactions.filter(t => {
    const date = new Date(t.transaction_date);
    return date >= new Date(startDate) && date <= new Date(endDate);
  });
}
```

- Преобразует дату каждой транзакции в объект Date.
- Фильтрует транзакции, которые попадают между startDate и endDate.
- Возвращает массив транзакций в указанном диапазоне дат.

```
function getTransactionsByMerchant(transactions, merchantName) {
  return transactions.filter(t => t.merchant_name === merchantName);
}
```

- Фильтрует транзакции по названию магазина/мерчанта.
- Возвращает массив транзакций с указанным мерчантом.

```
function calculateAverageTransactionAmount(transactions) {
  if (transactions.length === 0) return 0;
  return calculateTotalAmount(transactions) / transactions.length;
}
```

- Вычисляет среднюю сумму транзакции.
- Если транзакций нет, возвращает 0.
- Возвращает среднее значение суммы транзакции.

```
function getTransactionsByAmountRange(transactions, min, max) {
  return transactions.filter(t =>
    t.transaction_amount >= min && t.transaction_amount <= max
  );
}
```

- Фильтрует транзакции, сумма которых находится между min и max.
- Возвращает массив транзакций в заданном диапазоне сумм.

```
function calculateTotalDebitAmount(transactions) {
  return transactions
    .filter(t => t.transaction_type === "debit")
    .reduce((sum, t) => sum + t.transaction_amount, 0);
}
```

- Сначала выбирает только транзакции типа debit.
- Потом суммирует их суммы.
- Возвращает общую сумму всех дебетовых транзакций.

```
function findMostTransactionsMonth(transactions) {
  const counts = {};

  transactions.forEach(t => {
    const month = new Date(t.transaction_date).getMonth() + 1;
    counts[month] = (counts[month] || 0) + 1;
  });

  return Object.keys(counts).reduce((a, b) =>
    counts[a] > counts[b] ? a : b
  );
}
```

Считает, в каком месяце было больше всего транзакций.

- getMonth() возвращает 0–11, поэтому +1 чтобы получить 1–12.
- Возвращает номер месяца с наибольшим количеством транзакций.

```javascript
// 11
function findMostDebitTransactionMonth(transactions) {
  const counts = {};

  transactions
    .filter(t => t.transaction_type === "debit")
    .forEach(t => {
      const month = new Date(t.transaction_date).getMonth() + 1;
      counts[month] = (counts[month] || 0) + 1;
    });

  return Object.keys(counts).reduce((a, b) =>
    counts[a] > counts[b] ? a : b
  );
}
```

- Считает, в каком месяце было больше всего debit.
- Возвращает месяц с наибольшим количеством дебетовых транзакций.

```javascript
// 12
function mostTransactionTypes(transactions) {
  let debit = 0, credit = 0;

  transactions.forEach(t => {
    if (t.transaction_type === "debit") debit++;
    else if (t.transaction_type === "credit") credit++;
  });

  if (debit > credit) return "debit";
  if (credit > debit) return "credit";
  return "equal";
}
```

- Считает количество debit и credit.
- Сравнивает их и возвращает тип, которого больше, или "equal", если одинаково.
- Возвращает тип транзакций с наибольшим количеством.

```
function getTransactionsBeforeDate(transactions, date) {
  return transactions.filter(t =>
    new Date(t.transaction_date) < new Date(date)
  );
}
```

- Фильтрует транзакции, которые произошли до указанной даты.
- Возвращает массив транзакций до определённого дня.

```
function findTransactionById(transactions, id) {
  return transactions.find(t => t.transaction_id === id);
}
```

- Находит транзакцию по её id.
- find возвращает **первый найденный объект** или undefined, если не найден.

```
function mapTransactionDescriptions(transactions) {
  return transactions.map(t => t.transaction_description);
}
```

- Берёт все описания транзакций и делает из них массив строк.
- Возвращает массив описаний транзакций.

Тестирование функций:

```
console.log("Уникальные типы:", getUniqueTransactionTypes(transactions));

console.log("Общая сумма:", calculateTotalAmount(transactions));


console.log("Debit транзакции:",
  getTransactionByType(transactions, "debit")
);

console.log("Диапазон дат:",
  getTransactionsInDateRange(transactions, "2024-03-01", "2024-03-31")
);

console.log("По магазину Zara:",
  getTransactionsByMerchant(transactions, "Zara")
);

console.log("Среднее:",
  calculateAverageTransactionAmount(transactions)
);

console.log("Диапазон суммы:",
  getTransactionsByAmountRange(transactions, 100, 300)
);

console.log("Сумма debit:",
  calculateTotalDebitAmount(transactions)
);
```

```
[Running] node "/Users/nikita/Documents/js/lab3.js"
Уникальные типы: [ 'debit', 'credit' ]
Общая сумма: 225
Debit транзакции: [
  {
    transaction_id: '1',
    transaction_date: '2019-01-01',
    transaction_amount: 100,
    transaction_type: 'debit',
    transaction_description: 'Payment for groceries',
    merchant_name: 'SuperMart',
    card_type: 'Visa'
  },
  {
    transaction_id: '3',
    transaction_date: '2019-01-03',
    transaction_amount: 75,
    transaction_type: 'debit',
    transaction_description: 'Dinner with friends',
    merchant_name: 'RestaurantABC',
    card_type: 'Amex'
  }
]
Диапазон дат: []
По магазину Zara: []
Среднее: 75
Диапазон суммы: [
  {
    transaction_id: '1',
    transaction_date: '2019-01-01',
    transaction_amount: 100,
    transaction_type: 'debit',
    transaction_description: 'Payment for groceries',
    merchant_name: 'SuperMart',
    card_type: 'Visa'
  }
]
Сумма debit: 175
```

```
До даты: [
  {
    transaction_id: '1',
    transaction_date: '2019-01-01',
    transaction_amount: 100,
    transaction_type: 'debit',
    transaction_description: 'Payment for groceries',
    merchant_name: 'SuperMart',
    card_type: 'Visa'
  },
  {
    transaction_id: '2',
    transaction_date: '2019-01-02',
    transaction_amount: 50,
    transaction_type: 'credit',
    transaction_description: 'Refund for returned item',
    merchant_name: 'OnlineShop',
    card_type: 'MasterCard'
  },
  {
    transaction_id: '3',
    transaction_date: '2019-01-03',
    transaction_amount: 75,
    transaction_type: 'debit',
    transaction_description: 'Dinner with friends',
    merchant_name: 'RestaurantABC',
    card_type: 'Amex'
  }
]
```

```
Описания: [
  'Payment for groceries',
  'Refund for returned item',
  'Dinner with friends'
]
```

Пустой массив транзакций:

```
Уникальные типы: []
Общая сумма: 0
Debit транзакции: []
Диапазон дат: []
По магазину Zara: []
Среднее: 0
Диапазон суммы: []
Сумма debit: 0
```

Одна транзакция в массиве:

```
Уникальные типы: [ 'debit' ]
Общая сумма: 75
Debit транзакции: [
  {
    transaction_id: '3',
    transaction_date: '2019-01-03',
    transaction_amount: 75,
    transaction_type: 'debit',
    transaction_description: 'Dinner with friends',
    merchant_name: 'RestaurantABC',
    card_type: 'Amex'
  }
]
Диапазон дат: []
По магазину Zara: []
Среднее: 75
Диапазон суммы: []
Сумма debit: 75
Самый активный месяц: 1
Самый debit месяц: 1
Каких больше: debit
До даты: [
  {
    transaction_id: '3',
    transaction_date: '2019-01-03',
    transaction_amount: 75,
    transaction_type: 'debit',
    transaction_description: 'Dinner with friends',
    merchant_name: 'RestaurantABC',
    card_type: 'Amex'
  }
]
```

Вывод:

В ходе лабораторной работы была разработана программа на языке JavaScript для анализа транзакций с использованием массивов и функций высшего порядка. Программа позволяет получать статистические данные, фильтровать транзакции по типу, дате, сумме и мерчанту, а также вычислять общую и среднюю сумму операций.

Реализованные функции, такие как getUniqueTransactionTypes, calculateTotalAmount, getTransactionByType, getTransactionsInDateRange, позволяют эффективно обрабатывать данные и анализировать финансовую активность. Функции findMostTransactionsMonth и

findMostDebitTransactionMonth помогают определить месяцы с наибольшей активностью, а mostTransactionTypes показывает преобладающий тип транзакций.

Также предусмотрены функции для поиска конкретной транзакции по ID и для получения всех описаний транзакций, что упрощает проверку отдельных операций и визуальный анализ данных.

Вопросы:

1. Для обработки массивов объектов в JavaScript часто используют методы: **forEach** — для перебора элементов; **map** — для создания нового массива с преобразованными значениями; **filter** — для выборки объектов по условию; **reduce** — для вычислений и агрегирования данных; **find** — для поиска первого подходящего объекта; **some** и **every** — для проверки условий; **sort** — для сортировки объектов по свойствам.

2. В JavaScript строки с датами можно сравнивать напрямую через < или >, если они в формате ISO (YYYY-MM-DD), потому что порядок символов совпадает с хронологическим. Более надёжный способ — преобразовать строку в объект Date с помощью new Date(dateString) и затем сравнивать даты. Например, new Date("2023-03-31") < new Date("2023-04-01") вернёт true.

3. map() — создаёт новый массив, преобразуя элементы; filter() — создаёт новый массив, выбирая только подходящие элементы; reduce() — сводит массив к одному значению (сумме, объекту или другому результату).